

Forcing structured output such as JSON

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 8 of 12 · English notes

If you only say “give me JSON,” Claude often returns JSON *plus* a helpful paragraph (“Here is the JSON, want anything else?”). Downstream `json.loads` then dies. The 19 August pattern was **prompt engineering specific to Claude**: a user instruction that names the schema, an **assistant prefill** that opens a fenced JSON block, and a **stop sequence** on the closing fence so generation stays inside the object.

The failure mode

Natural-language models like to be polite. Leading and trailing prose is fatal if a service expects a single object with quoted strings (Vijay’s “name: Vijay” example). Structured consumers — agents writing files, AWS config generators, APIs — need **only** the payload.

Prefill as a seed, not a second pass

Classroom prefill looked like an extra message: `role: assistant`, content starting with a markdown JSON fence (spoken as ticks + `json`). The user message was e.g. “List 5 popular programming languages and their main use cases in JSON format.” The model continues from the fence, so the next characters are expected to be `{ ...`. The stop sequence is the closing fence. Mritunjay asked whether the server generates everything and then “translates” or truncates: **no** — it is how the prompt is assembled so decoding starts at the seed. Tokens billed are the continuation, not a discarded preamble.

Multiple fenced languages in one answer

A harder demo asked for one user-config sample in **bash**, **XML**, and **JSON**, each in its own fenced block, with **no explanations between blocks**, ending at a marker such as `END` that also sat in `stop_sequences`. That is few-shot / example-shaped instruction: show the exact fence labels you want. Agents that must write files love this contract.

Technique	Strength	Weakness
Instruction only (“reply with JSON”)	Simple	Preamble/prose leaks
Prefill + stop on fences	Works on models without schema mode; taught in class	Fragile if the model emits a different fence or extra keys
Official structured outputs / JSON schema	Constrained decode when your model supports it	Must match current API; still validate
Pydantic / JSON Schema in your code	Validates types after the fact	Does not by itself stop prose unless combined with the above

Validate structure in code, not always with another LLM

Later the same day, evals: if the contract is “JSON object with name: string and age: number,” a **Pydantic** (or similar) model is enough for shape. An LLM-as-judge is overkill and expensive. Tools or retrieval judge *values* (is this age correct in the HR system?). Keep those layers distinct.

What to put in the user content

- Name the format and the keys.
- Forbid extra keys / markdown if you are not using fences.
- Give one compact example (few-shot) if the shape is nested.
- Keep temperature low for extraction-style JSON.

PRODUCTION 2026

Class trick = prefill + stop. Current Anthropic guidance also documents **structured outputs**. For UST delivery, prefer schema-constrained APIs when the model supports them, and still parse-validate.

Exam: be able to explain the prefill pattern Vijay coded.

XML as an alternative envelope

If the organisation already thinks in XML, fences can be XML tags instead of markdown. Claude prompting docs emphasise XML-style tags as delimiters (next chapter). JSON vs XML is a consumer choice; the control pattern (clear envelope + stop) is the same.

DOCS OVER LIVE CLASS JSON MODE

Any “JSON mode” toggle in a console UI may have moved. Trust **structured outputs** and the Messages API.

Official documentation

- **Structured outputs**
<https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
 Schema-constrained JSON on supported models.
- **Messages API**
<https://docs.claude.com/en/api/messages>
 Assistant prefill and stop_sequences used in the lab.
- **Prompt engineering overview**
<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>
 Output format instructions and examples.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.